

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Deep Learning based medical image segmentation — Part 5 — Data Preprocessing



Diya S

Following ▾

7 min read · Jul 6, 2023



74



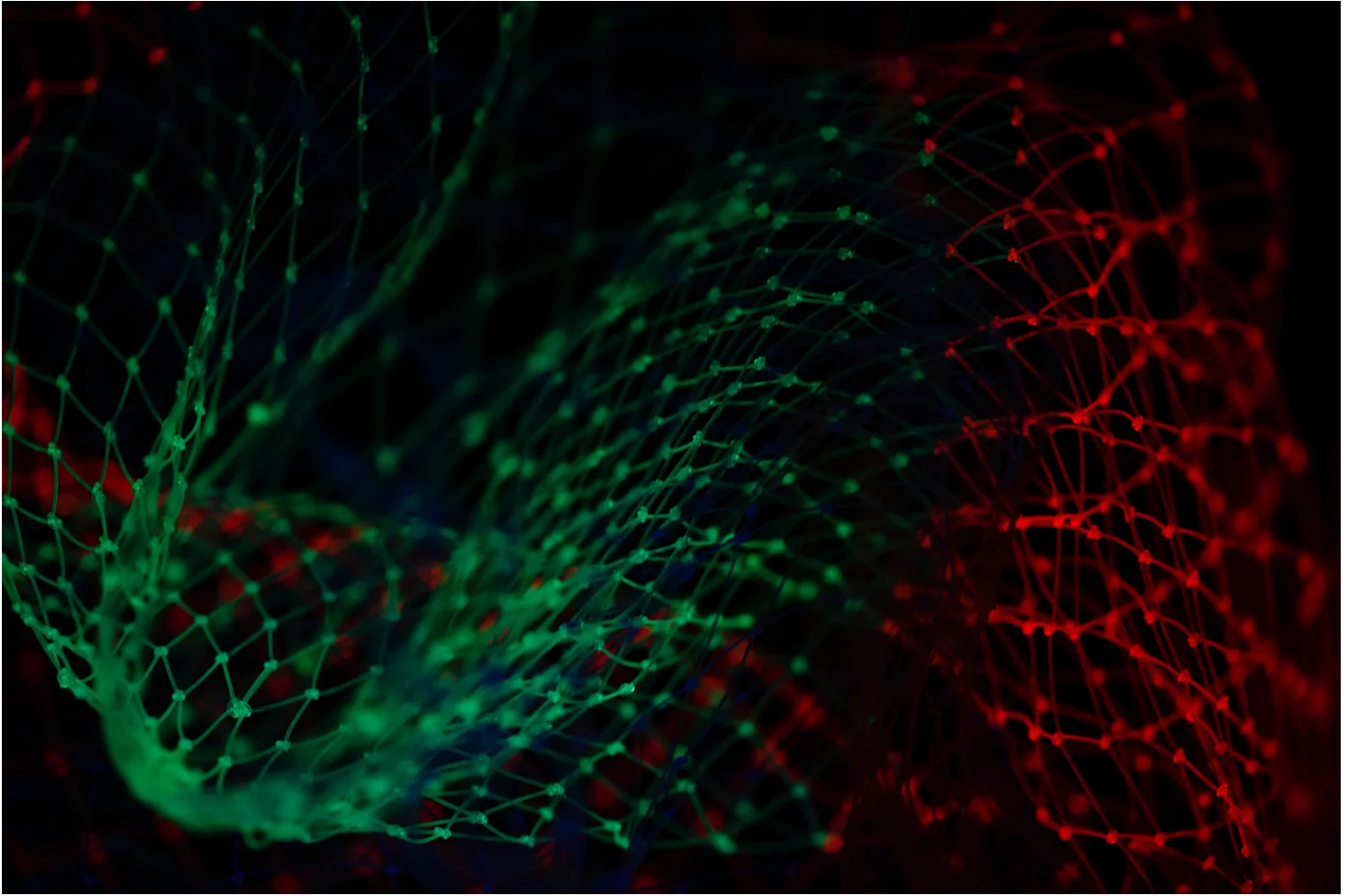


Photo by [Pietro Jeng](#) on [Unsplash](#)

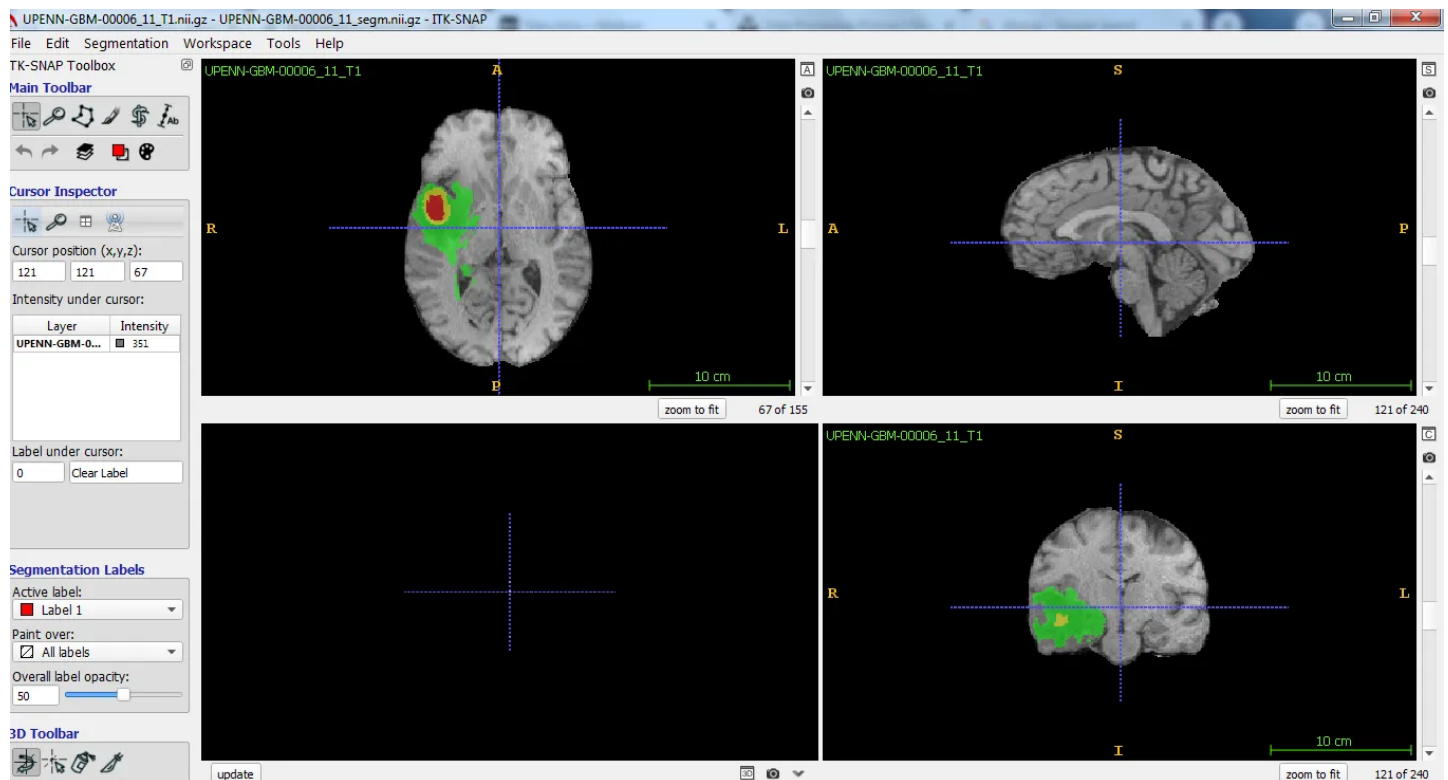
Before we begin training the deep learning model, we need to preprocess the MRI image and labels dataset. Most of this work, such as image target spacing, image resampling, and intensity normalization is performed for us by nnUNet. However, on the segmentation maps (label files), nnUNet expects consecutive integers for labels. We will need to look at the training label files that we downloaded from TCIA and see if anything needs to be changed.

Segmentation Map (Label) File Preprocessing

Let's take a look at a tumor segmentation file. For this, we will use the ITKSnap software. I am doing this on my Windows laptop.

After installing ITKSnap, open a training image by **File** → **Open Main Image**. Here, I opened *images_structural\UPENN-GBM-00006_11\UPENN-GBM-00006_11_T1.nii.gz*

Now apply the segmentation file on top by **Segmentation** → **Open Segmentation**. Here I opened patient # 6's ground truth tumor segmentation file *images_seg\UPENN-GBM-00006_11_seg.nii.gz*



ITKSnap Tumor Segmentation Map. Image by author

Reading the [paper published with this dataset](#)¹, we know that there are 3 labels —

1. Necrotic Tumor Core (NCR) — Red area in the image
2. Enhancing Tumor (ET) — Yellow area in the image
3. Peritumoral Edematous/Infiltrated tissue (ED) — Edema for short, green area in the image

Each of these areas will have an integer value associated with it, and nnUNet requires them to be consecutive integers, with the black background having a value of zero.

Now, let's open the segmentation file with Python and see what the integer labels for each of these segmented areas are.

- Open a new Colab notebook — *02_t501_glio_process_labels.ipynb*
- Mount the Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

- Import the required packages. We will use *nibabel* package to process the segmentation files.

```
import nibabel as nib
import numpy as np
import os
import pathlib
```

- Set up the file path and make sure you can access the segmentation files

```
base_img_path = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Gliob
tr_label_files = os.listdir(base_img_path)
tr_label_files.sort()
# print(tr_label_files)
print(len(tr_label_files))
```

- Check the label values for each training image. This is going to print the min and max label values, and also the actual unique label values.

```
for file in tr_label_files:

    file_name = base_img_path + file
    # Load image
    img = nib.load(file_name)
    print("-"*100)
    print ("Patient label file: ", file)

    # Store image as a numpy array
    img_data = img.get_fdata()

    # Check array min, max and unique values for image label values
    print("Before label check")
```

```
print(np.amin(img_data),np.amax(img_data))
print(np.unique(img_data))
```

```
-----
Patient label file: 9.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
```

```
-----
Patient label file: 21.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
```

```
-----
Patient label file: 29.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
-----
```

- As we can see, the unique label values are 0, 1, 2 and 4. Since nnUNet needs these values to be consecutive, it will fail unless we update the label value 4 to 3. We can do that using nibabel. We will read the segmentation file, update the label value 4 to 3, and write the file back to our **labelsTr** folder.

```
for file in tr_label_files:

    file_name = base_img_path + file
    # Load image
    img = nib.load(file_name)
    print("-"*100)
```

```

print ("Patient label file: ", file)

# Store image as a numpy array
img_data = img.get_fdata()

# Check array min, max and unique values for image
print("Before label check")
print(np.amin(img_data),np.amax(img_data))
print(np.unique(img_data))

# Where label is 4, reset it to 3 so nnUnet doesnt complain about non-consecu
img_data[(img_data == 4.0)] = 3

# Check min, max and unique values again
print("After label conversion")
print(np.amin(img_data),np.amax(img_data))
print(np.unique(img_data))

# Convert array back to Nii image
new_img = nib.Nifti1Image(img_data, img.affine, img.header)

# Set up new image path
new_img_path = base_img_path + file
print ('Saved to: ' + new_img_path)
# Save image to out_dir_path
nib.save(new_img, new_img_path)

```

```

-----
Patient label file: 100.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
After label conversion
0.0 3.0
[0. 1. 2. 3.]
Saved to: /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma
-----

Patient label file: 102.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
After label conversion

```

```
0.0 3.0
[0. 1. 2. 3.]
Saved to: /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma
-----
Patient label file: 105.nii.gz
Before label check
0.0 4.0
[0. 1. 2. 4.]
After label conversion
0.0 3.0
[0. 1. 2. 3.]
Saved to: /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma
-----
```

- We are now done with the label file preprocessing and can move on to the nnUNet preprocessing steps.

Generating metadata about our dataset

- Before we begin using nnUNet, we need to create a metadata (information about the data) file that will provide nnUNet with some basic configuration parameters. This file has to be called *dataset.json*. The nnUNet package provides a handy utility to generate this. Let's generate this file now.
- Open a new Colab notebook — *03_t501_glio_gen_dataset_json.ipynb*
- Mount your Google Drive


```
from google.colab import drive
drive.mount('/content/drive')
```

- Install the **batchgenerators** Python package, which is required by nnUNet. This is a pip command we are running on the Google Colab virtual machine (VM), so it needs to be prefixed by an !

```
!pip install batchgenerators
```

- Install the nnUNet V2 package

```
!pip install nnunetV2
```

- Setup the parameters required by the metadata file generation utility. We will provide the channel names for our MRI sequences, and also the label value mapping information.

```
# Set up all the parameters required by the metadata generation utility

output_folder = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Gliob
channel_names = {0: 'T1', 1: 'T1GD', 2: 'T2', 3: 'FLAIR'}
labels = {'background': 0, 'ET': 1, 'NCR': 2, 'ED': 3}
```

```

file_ending = '.nii.gz'
region_class_order = {
    'background': 0,
    'whole tumor': (1, 2, 3),
    'tumor core': (2, 3),
    'enhancing tumor': 3
}
num_training_cases = 117
dataset_name = 'UPENN-GBM'
description = 'UPENN-GBM tumor segmentation'

```

- Import the metadata generation utility and run it to produce the `dataset.json` file

```

from nnunetv2.dataset_conversion.generate_dataset_json import generate_dataset_
generate_dataset_json(output_folder=output_folder, channel_names=channel_names,
                      file_ending=file_ending, region_class_order=region_class_o
                      num_training_cases=num_training_cases, dataset_name=datas

```

- You should now see a `dataset.json` file in **MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma** folder. It should look something like this.

```

{
  "channel_names": {
    "0": "T1",
    "1": "T1GD",
    "2": "T2",
    "3": "FLAIR"
  }
}

```

```

    },
    "labels": {
        "background": 0,
        "ET": 1,
        "NCR": 2,
        "ED": 3
    },
    "numTraining": 117,
    "file_ending": ".nii.gz",
    "name": "UPENN-GBM",
    "description": "UPENN-GBM tumor segmentation",
    "region_class_order": {
        "background": 0,
        "whole tumor": [
            1,
            2,
            3
        ],
        "tumor core": [
            2,
            3
        ],
        "enhancing tumor": 3
    }
}

```

Verifying dataset integrity and preprocessing

- nnUNet checks all the required folder and file configurations are appropriate before model training. It also preprocesses the images to standardize and normalize them before model training. This is performed by the **nnUNet_plan_and_preprocess** utility. The first time you run this, you should also check the dataset integrity.

- Open a new Colab notebook — *04_t501_glio_verify_data_integrity.ipynb*
- Mount your Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

- Install the nnUNet v2 package. Since the Colab notebook attaches to a new VM, we have to install this package every time we create a new notebook, or update the runtime type.

```
!pip install nnunetv2
```

- Setup the environment variables and run the dataset verification and preprocessing. Remember, the environment variables only apply to the context of this cell. It is not saved or apply to subsequent cells or commands.

```
import os
os.environ['nnUNet_raw'] = "/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw"
os.environ['nnUNet_preprocessed'] = "/content/drive/MyDrive/TCIA/nnUNet/nnUNet_preprocessed"
os.environ['nnUNet_results'] = "/content/drive/MyDrive/TCIA/nnUNet/nnUNet_results"

# Verify dataset integrity has to be executed only the first time you are pre-p
```

After successful plan and preprocessing

```
!nnUNetv2_plan_and_preprocess -d 501 --verify_dataset_integrity
```

```

Fingerprint extraction...
Dataset501_Glioblastoma
Using <class 'nnunetv2.imageio.simpleitk_reader_writer.SimpleITKIO'> as reader/

#####
verify_dataset_integrity Done.
If you didn't see any error messages then your dataset is most likely OK!
#####

Using <class 'nnunetv2.imageio.simpleitk_reader_writer.SimpleITKIO'> as reader/
100% 117/117 [02:29<00:00, 1.28s/it]
Experiment planning...
2D U-Net configuration:
{'data_identifier': 'nnUNetPlans_2d', 'preprocessor_name': 'DefaultPreprocessor'}

Using <class 'nnunetv2.imageio.simpleitk_reader_writer.SimpleITKIO'> as reader/
3D fullres U-Net configuration:
{'data_identifier': 'nnUNetPlans_3d_fullres', 'preprocessor_name': 'DefaultPreprocessor'}

Plans were saved to /content/drive/MyDrive/TCIA/nnUNet/nnUNet_preprocessed/Data
Preprocessing...
Preprocessing dataset Dataset501_Glioblastoma
Configuration: 2d...
100% 117/117 [05:28<00:00, 2.81s/it]
Configuration: 3d_fullres...
100% 117/117 [05:25<00:00, 2.78s/it]
Configuration: 3d_lowres...
INFO: Configuration 3d_lowres not found in plans file nnUNetPlans.json of dataset

```

- If you've followed along the journey so far, set up all your folders and files as described in the earlier articles, hopefully you will see a message like the above that your dataset integrity is most likely OK. If there are issues like file names or folder names not matching, or training images

that do not have corresponding label files, this step will fail, and you will

Medium

🔍 Search

✍ Write



- After verifying the dataset integrity, nnUNet will produce the training plans for 2d, 3d_fullres, and 3d_lowres configurations. Here, it skipped the 3d low resolution configuration due to the relatively small number of training images.

This was a long post, but I hope you are able to follow along. We are ready to begin model training now. We can sit back and let nnUNet do the work. We will continue the process in our next post.

Python

Artificial Intelligence

Neural Networks

Radiology

Brain Tumor



Written by Diya S

92 followers · 14 following

Following ▾

I am a high school student on a journey to understand how disruptive technologies can transform and democratize healthcare in the coming decade
D.S

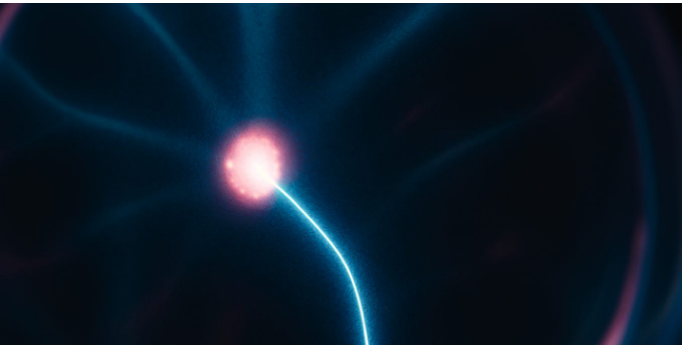
No responses yet



Charlie Chien

What are your thoughts?

More from Diya S



Diya S

Glioblastoma brain tumor segmentation—Part 6— Neural...

Train a deep neural network model on Glioblastoma brain tumor multimodal MRI...

Jul 6, 2023

33

1



Diya S

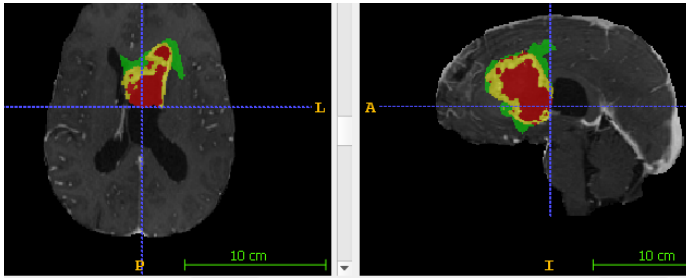
Glioblastoma brain tumor segmentation—Part 4— Data...

Setting up TCIA Glioblastoma data for nnUNet brain tumor segmentation

Jul 5, 2023

41





 Diya S

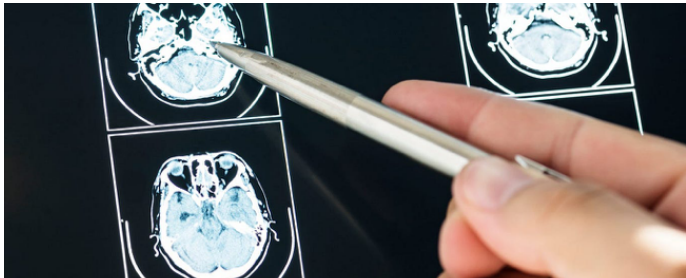
Glioblastoma brain tumor segmentation—Part 1—Getting...

Start our automated brain tumor segmentation journey by downloading and...

May 14, 2023

 21



 Diya S

Glioblastoma brain tumor segmentation—Part 3—Preparin...

In our previous post, we downloaded the UPenn Glioblastoma MRI dataset from TCIA...

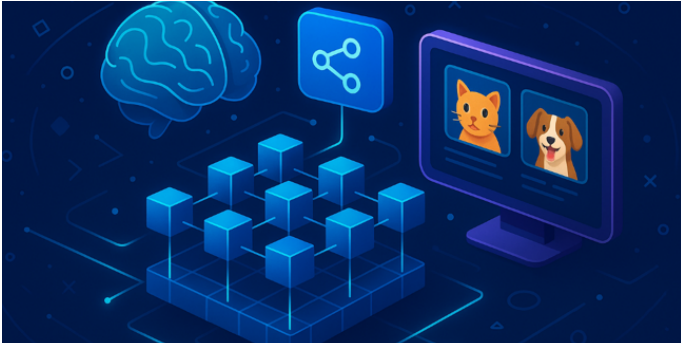
Jun 19, 2023

 5

See all from Diya S

Recommended from Medium

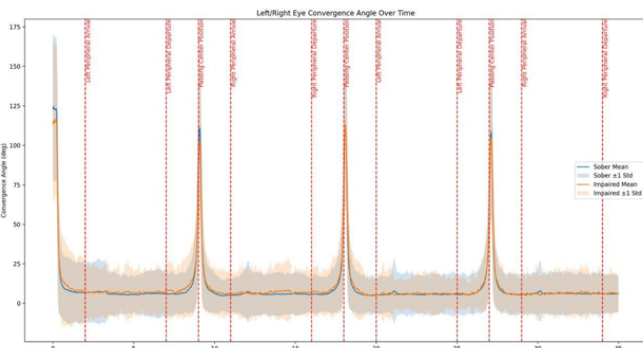


Old Noisy Speaker

How AI Learn to See: Building a CNN in Python

Introduction: Teaching Machines to See

★ May 10

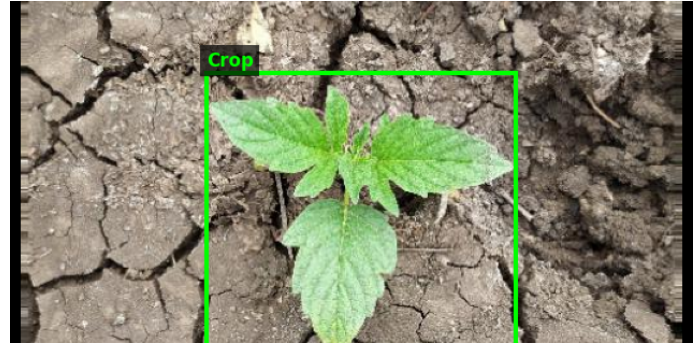


Siddhant Nitin

Eye-Track-Analysis: Building an Impairment Detection System wi...

What if your eyes could speak—not just in expressions, but in data? In my latest side...

★ Apr 16 🖱️ 4

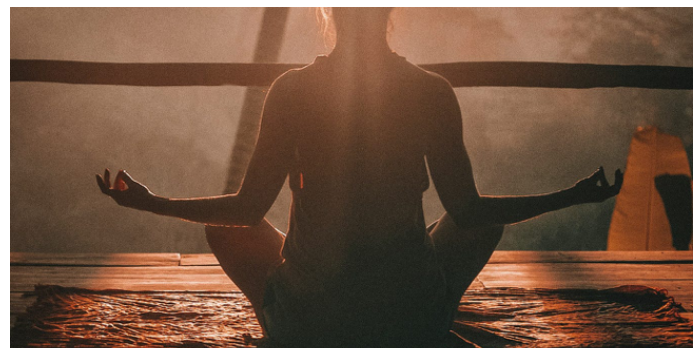


Brian Hulela

Understanding Image Annotation Formats for Object Detection

A Beginner's Guide to YOLO, COCO, Pascal VOC, and More

★ Apr 1



In Reclabs Inc by Odemakinde Elisha

Are 3D-CNN-LSTM Vision Encoders powerful for Video...

Have you ever wondered how GPTs today handle video understanding? Here is a...

Mar 24 🖱️ 6





 Nikulsinh Rajput

This Pandas One-Liner Saved Me Hours of Data Cleaning

It was hiding in plain sight—and now I use it in every project.

★ Jun 18 🖱 1



 Toufiq Musah

Introduction to TorchIO for 3D MRI Processing: Augmentation...

In Part 1 of the Introduction to Torchio tutorial, we looked at how to perform...

May 11 🖱 70 💬 3



See more recommendations